UNIT 3->

STACK

A stack is a list of elements in which an element may be inserted or deleted only at one end, called the top of the stack.

Stack is linear type data structure it is called **last in first out(lifo)**. In this structure insertion and deletion can take place only at one end called **top of the list**.

There are 2 basic operations associated with a stack.

They are

- **PUSH**- is the term used to insert an element into a stack.
- **Pop**- is the term used to delete an element from stack.

REPRESENTATION OF STACKS

Stacks may be represented in the computer in various ways,

- Array representation of stack
- Linked representation of stack

ARRAY REPRESENTATION OF STACK

- In computer's memory, stacks can be represented as a linear array.
- Every stack has a variable called top associated with it, which is used to store the address of the topmost element of the stack.
- It is this position where the element will be added to or deleted from
- There is another variable called MAX, which is used to store the maximum number of elements that the stack can hold.
- The condition top=null will indicate that the stack is empty
- If top=max-1,then the stack is full.

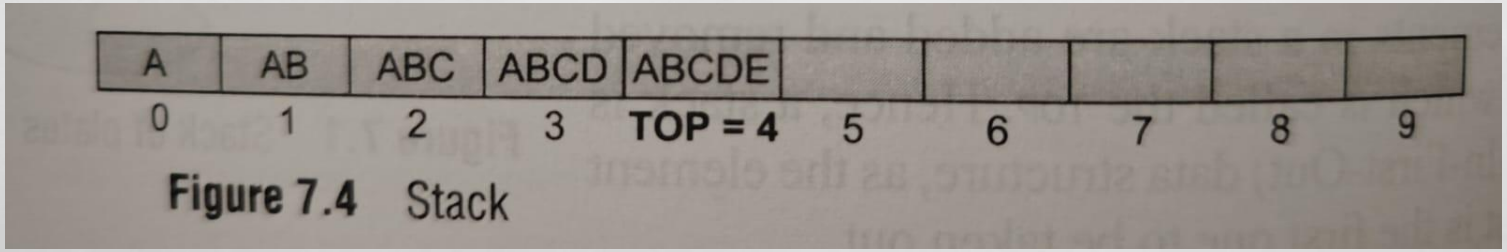


Figure 7.4 Stack

OPERATIONS ON STACK

Stack supports 2 basic operations: PUSH & POP .

PUSH OPERATION

- The push operation adds an element to the top of the stack .
- Before inserting the value, we must first check if TOP=MAX-1, if that is the case then the stack is full and no more insertion can be done.
- If an attempt is made to insert a value in a stack, an overflow message will be printed.

Algorithm to insert an element in a stack

```
Step 1: IF TOP = MAX-1
            PRINT "OVERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET TOP = TOP + 1
Step 3: SET STACK[TOP] = VALUE
Step 4: END
```

Figure 7.7 Algorithm to insert an element in a stack

Pop Operation

- The pop operation is used to delete the topmost element from the stack.
- Before deleting the value, we must first check if TOP=NULL because if that is the case, then it means the stack is empty and no more deletions can be done.
- If an attempt is made to delete a value from a stack that is already empty, an UNDERFLOW message is printed

Algorithm to delete an element from a stack

```
Step 1: IF TOP = NULL
            PRINT "UNDERFLOW"
            Goto Step 4
        [END OF IF]
Step 2: SET VAL = STACK[TOP]
Step 3: SET TOP = TOP - 1
Step 4: END
```

Figure 7.10 Algorithm to delete an element from a stack

LINKED REPRESENTATION OF STACK

- The drawback of using array implementation of stack is that the array must be declared to have some fixed size.
- In case the stack is a very small one or its maximum size is known in advance, then the array implementation of the stack gives an efficient implementation.
- But if the array size cannot be determined in advance, then the other alternative, i.e., Linked representation, is used.
- The start pointer of the linked list is used as top. All insertions and deletions are done at the node pointed by TOP. If TOP = NULL, then it indicates that the stack is empty.

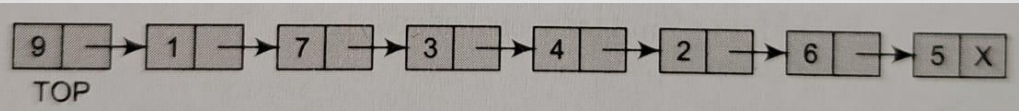


Figure 7.13 Linked stack

OPERATIONS ON A LINKED STACK

LINKED STACK SUPPORTS 2 BASIC OPERATIONS :

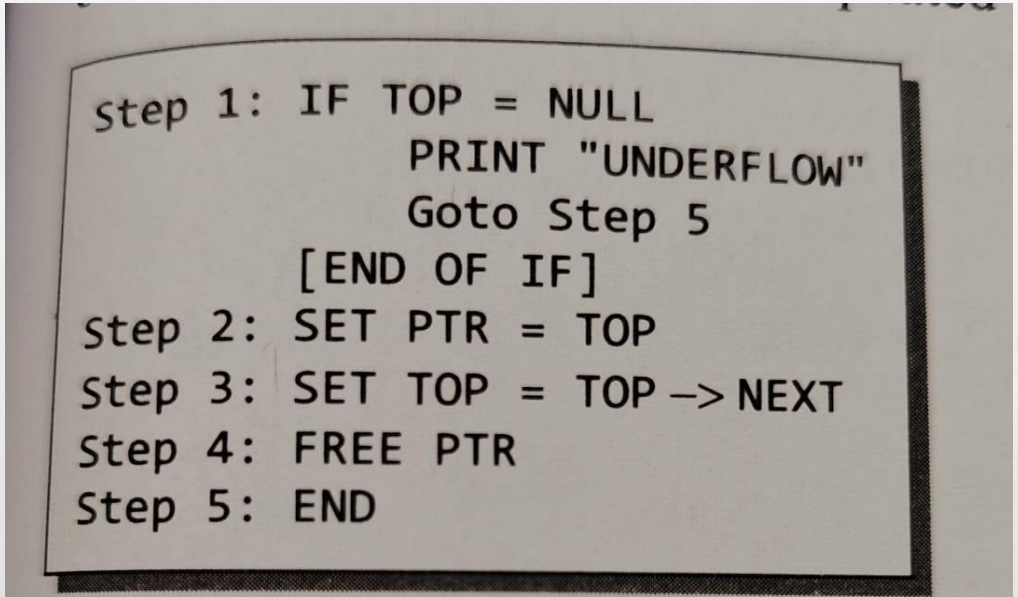
- PUSH OPERATION
- POP OPERATION

Algorithm to insert an element in a linked stack

```
Step 1: Allocate memory for the new
        node and name it as NEW_NODE
Step 2: SET NEW_NODE -> DATA = VAL
Step 3: IF TOP = NULL
            SET NEW_NODE -> NEXT = NULL
            SET TOP = NEW_NODE
        ELSE
            SET NEW_NODE -> NEXT = TOP
            SET TOP = NEW_NODE
        [END OF IF]
Step 4: END
```

Figure 7.16 Algorithm to insert an element in a linked stack

ALGORITHM TO DELETE AN ELEMENT FROM A LINKED STACK



```
Step 1: IF TOP = NULL
            PRINT "UNDERFLOW"
            Goto Step 5
        [END OF IF]
Step 2: SET PTR = TOP
Step 3: SET TOP = TOP -> NEXT
Step 4: FREE PTR
Step 5: END
```

Figure 7.19 Algorithm to delete an element from a linked stack

APPLICATIONS OF STACKS

- Stacks can be used for expression evaluation.(infix , postfix &prefix)
- Function call management (recursion):
- Used for memory management.
- Reversing a List.
- Undo/redo operations:
- Stacks can be used for conversion from one form of expression to another.
- Palindrome checking:
- Stacks can be used to check parenthesis matching in an expression.

ARITHMETIC EXPRESSION;

- An **arithmetic expression** consists of constants and operations.That follows specific rules to produce a result.
- It can be represented in different notations (**infix, prefix, postfix**) and is often evaluated using **stacks** for efficient computation..

Infix notation

- The operator appears **between** operands.
- Example: a + b * c
- Requires parentheses or operator precedence rules to determine order.

Prefix notation (polish notation)

- The operator appears **before** operands.
- Example: + * a b c
- Eliminates the need for parentheses.

Postfix notation (Reverse Polish Notation - RPN)

- The operator appears **after** operands.
- Example: a b c * +
- Easy to evaluate using a stack, as they don’t require precedence rules.

Computer usually evaluates an arithmetic expression written in infix notation in two steps:

- First step: converts the infix expression to postfix notation.
- Second step: evaluates the postfix expression.

Stack is the main tool that is used to accomplish the given task

EVALUATION OF A POSTFIX EXPRESSION

This algorithm finds the VALUE of an arithmetic expression P written in postfix notation

1. Add a right parenthesis ")" at the end of P. [This acts as a sentinel.]

2. Scan P from left to right and repeat steps 3 and 4 for each element of P until the sentinel “)" is encountered.

3. If an operand is encountered, put it on STACK.

4. If an operator $\otimes$ is encountered, then:

(a) Remove the two top elements of STACK , where A is the top element and B is the next-to-top element.

(b) Evaluate B $\otimes$ A.

(c)Place the result of (b) back on STACK.

[End of if structure.]

[End of step 2 loop.]

5. Set VALUE equal to the top element on STACK.

6. Exit.

Example: Evaluate Postfix Expression 6 2 3 + * 4 –
Step1: 6 2 3 + * 4 –)
Step2:Scan from left to right

Symbol	Stack (After Operation)	Explanation
`6`	`6`	Push `6`
`2`	`6 2`	Push `2`
`3`	`6 2 3`	Push `3`
`+`	`6 5`	`2 + 3 = 5`, push `5`
`*`	`30`	`6 * 5 = 30`, push `30`
`4`	`30 4`	Push `4`
`-`	`26`	`30 - 4 = 26`, push `26`

Final Answer: `26`

- **Evaluate the following postfix expression showing the stack status.**

P:3,5,+,6,4,-,*,4,1,-,2,^,+

Step	Scanned Symbol	Operation	Stack	Explanation
1	3	Push 3	[3]	Push 3 onto the stack.
2	5	Push 5	[3, 5]	Push 5 onto the stack.
3	+	3 + 5	[8]	Pop 5 and 3, add them to get 8.
4	6	Push 6	[8, 6]	Push 6 onto the stack.
5	4	Push 4	[8, 6, 4]	Push 4 onto the stack.
6	-	6 - 4	[8, 2]	Pop 4 and 6, subtract to get 2.
7	*	8 * 2	[16]	Pop 2 and 8, multiply to get 16.
8	4	Push 4	[16, 4]	Push 4 onto the stack.
9	1	Push 1	[16, 4, 1]	Push 1 onto the stack.
10	-	4 - 1	[16, 3]	Pop 1 and 4, subtract to get 3.
11	2	Push 2	[16, 3, 2]	Push 2 onto the stack.
12	^	3 ^ 2	[16, 9]	Pop 2 and 3, raise 3 to the power of 2 to get 9.
13	+	16 + 9	[25]	Pop 9 and 16, add them to get 25.

Evaluate the following postfix expressions showing the stack status.

- **P:3,1,+,2,^,7,4,-,2,*,+,5,-**
- **P:5,6,2,+,*,1,2,4,/,^+**

Transforming infix expression into postfix expression

Algorithm POLISH(Q, P)

Suppose Q is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression P.

1. Push "(" onto STACK, and add “)“ to the end of Q .

2. Scan Q from left to right and repeat steps 3 to 6 for each element of Q until the STACK is empty:

3. If an operand is encountered, add it to P.

4. If a left parenthesis is encountered, push it onto STACK.

5. If an operator ⊗ is encountered, then:

(a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) which has the same precedence as or higher precedence than ⊗ .

(b) Add ⊗ to STACK.

[End of if structure.]

6. If a right parenthesis is encountered, then:

(a)repeatedly pop from STACK and add to P each operator (on the top of STACK) until a left parenthesis is encountered.

(b)remove the left parenthesis. [Do not add the left parenthesis to P].

[End of if structure.]

[End of step 2 loop.]

7. Exit.

eg:Infix: A + B * C

Step 1: add) parenthesis at the end

A+B*C)

Step	Read	Action	Postfix	Stack
1	A	Operand → add	A	
2	+	Operator → push	A	+
3	B	Operand → add	A B	+
4	*	Operator → push	A B	+ *
5	C	Operand → add	A B C	+ *
6	)	Pop stack to postfix	A B C * +	

- Infix: (A + B) * (C - D) / E

STEP1: (A + B) * (C - D) / E)

Step	Read	Action	Postfix	Stack
1	(	Push		(
2	A	Operand → add	A	(
3	+	Push	A	(+
4	B	Operand → add	A B	(+
5	)	Pop until '('	A B +	
6	*	Push	A B +	*
7	(	Push	A B +	* (
8	C	Operand → add	A B + C	* (
9	-	Push	A B + C	* (-
10	D	Operand → add	A B + C D	* (-
11	)	Pop until '('	A B + C D -	*
12	/	Pop higher/equal, push	A B + C D - *	/
13	E	Operand → add	A B + C D - * E	/
14	End	Pop remaining stack	A B + C D - * E /	

Queues:

A queue is a linear list of elements in which,

- deletion can take place only at one end called Front, and.

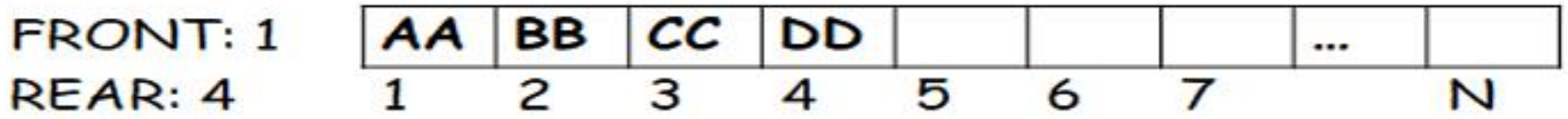
- Insertion takes place at one end called Rear .

Queues are also known as First-In- First-Out (FIFO) list

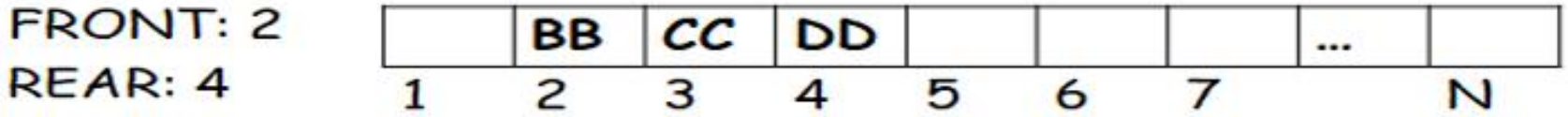
-the order in which elements enter a queue is the order in which they leave.

The condition FRONT=NULL will indicate that queue is empty.

Queue



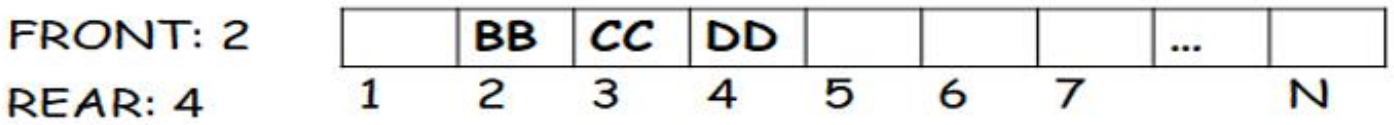
Delete an element



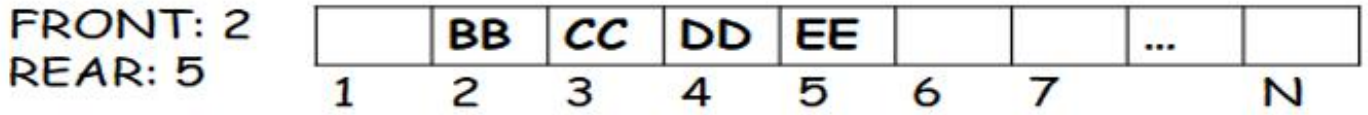
Whenever an element is deleted from the queue, the value of FRONT is increased by 1

FRONT = FRONT + 1

Queue



Insert an element



Whenever an element is inserted into the queue, the value of REAR is increased by 1

REAR = REAR + 1

Representation of queues:

- Array representation of Queues.
- Linked representation of Queues.

Algorithm to Insert an element in Queue using Arrays

Step 1: IF REAR=MAX-1

Write OVERFLOW

Goto step 4

[END OF IF]

Step 2: IF FRONT=-1 and REAR=-1

SET FRONT=REAR=O

ELSE

SET REAR=REAR +1

[END OF IF]

Step 3: SET QUEUE[REAR] = NUM

Step 4: EXIT

Algorithm to delete an element from Queue using Arrays

Step 1:IF FRONT=-1 OR FRONT>REAR

Write UNDERFLOW

ELSE

SET VAL=QUEUE[FRONT]

SET FRONT=FRONT +1

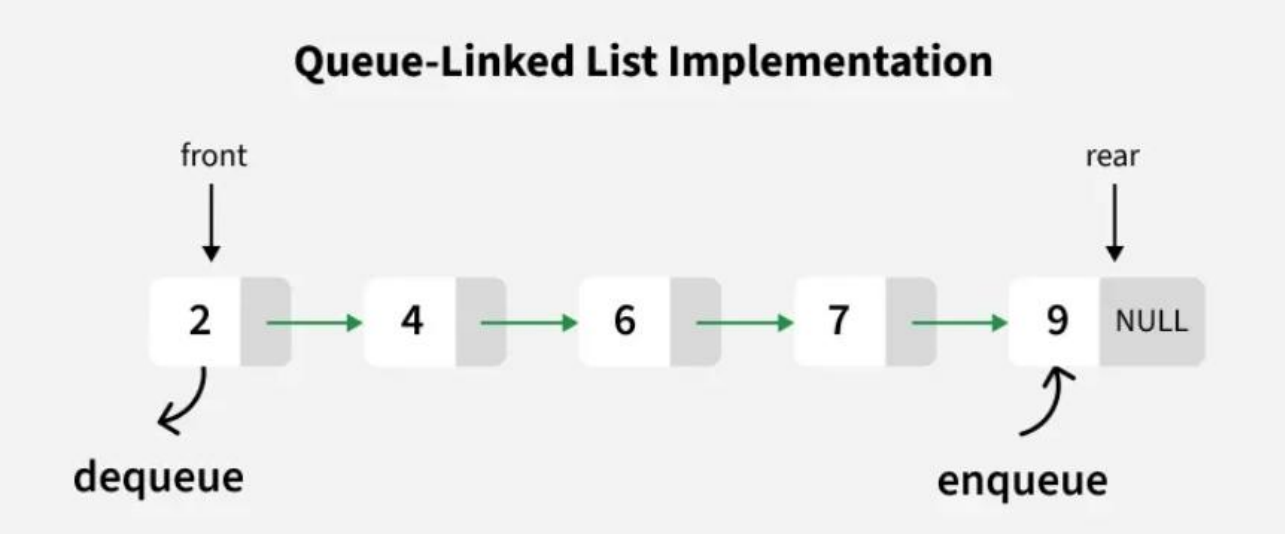
[END OF IF]

Step 2:EXIT

- **Enqueue**:The process of inserting an element in the queue is called Enqueue.
- **Dequeue:** The process of deleting an element from the queue is called Dequeue.

Linked list representation of QUEUE

- A linked queue is a queue implemented as a linked list with two pointer variables FRONT and REAR pointing to the nodes which is in the front and rear of the queue.
- The INFO field of the node holds the data in the queue and the NEXT is the pointer to the next element in queue.



Algorithm to insert an element in a linked Queue.

```
LINKQ_INSERT(INFO, LINK, FRONT, REAR, AVAIL, ITEM)

This procedure inserts an ITEM into A linked queue
1. [available space?] if AVAIL = NULL, then:
     write  OVERFLOW and Exit
2. [ Remove first node from AVAIL list]
     Set NEW:= AVAIL   and AVAIL : = LINK[AVAIL]
3. set INFO[NEW] :=  ITEM and LINK[NEW] = NULL
4. If (FRONT = NULL)then
          FRONT = REAR = NEW
           [if Queue is empty then ITEM is the first element in the queue Q]
           Else
           set    LINK[REAR] := NEW  and  REAR = NEW
            [REAR points to the new node appended to the end of the list]
5. Exit
```

Algorithm to delete an element from a linked queue

LINKQ_DELETE(INFO, LINK, FRONT, REAR, AVAIL, ITEM)
This procedure deletes the front element of the linked queue and stores it in \\
ITEM.
1. [Linked queue empty?]
   If (FRONT = NULL) then write: UNDERFLOW and EXIT
2. set TEMP = FRONT
3. ITEM = INFO(TEMP)
4. FRONT = LINK (TEMP)
5. LINK(TEMP) = AVAIL and AVAIL = TEMP
   [return the deleted node TEMP to the AVAIL list]
4. EXIT

Types of queue

- Simple queue
- Circular queue
- Double ended queue(deque)
- Priority queue

Circular Queue

- A **Circular Queue** is a linear data structure that follows the **FIFO (First In First Out)** principle, but it treats the array **as circular** — the end of the array connects back to the beginning.
- In a normal (linear) queue using an array, space is wasted after deletions.
- A circular queue **reuses space** by wrapping around when it reaches the end.
- It improves the efficiency of insertions and deletions.

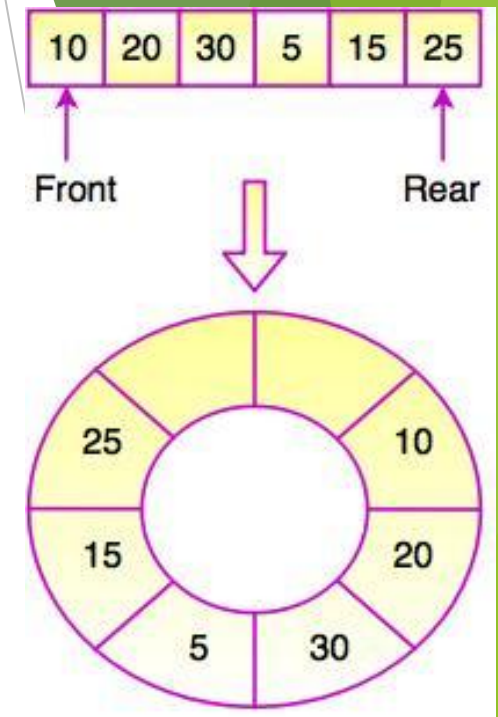
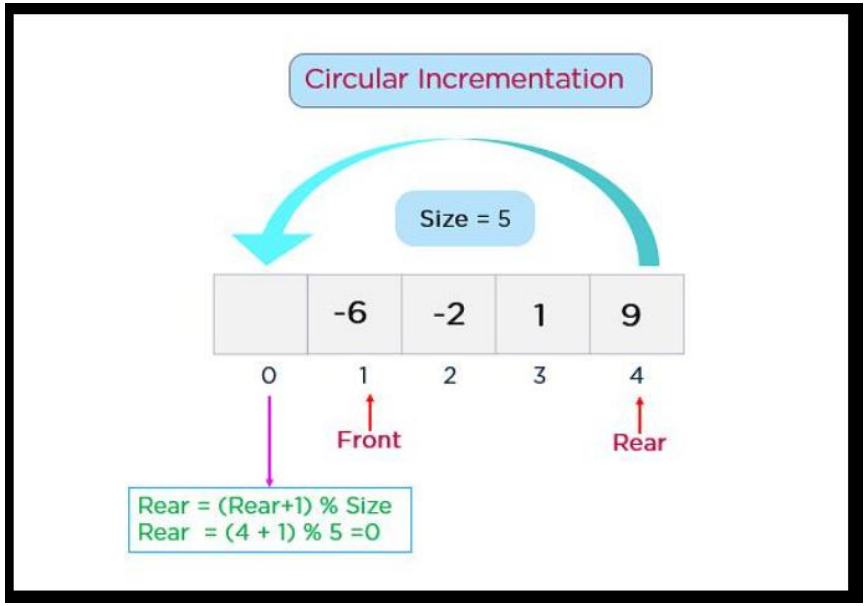


Fig. Circular Queue

Basic Operations:

1. **Enqueue (Insertion)**
   - Add element at the `rear` position.
   - Update `rear` as:
     `(rear + 1) % size`



2. **Dequeue (Deletion)**
    - Remove element from the `front` position.
    - Update `front` as:

      `(front + 1) % size`
3. **Display**
    - Print elements from `front` to `rear`, wrapping around if necessary.

Algorithm for Insertion (Enqueue) in Circular Queue Using Array

- Algorithm Enqueue(Queue, X, N)

```
1.  If ((Rear + 1) % N == Front) Then
      Print "Queue Overflow"
      Return
2.  If (Front == -1) Then   // First element insertion
      Front = 0
      Rear = 0
    Else
      Rear = (Rear + 1) % N
3.  Queue[Rear] = X
4.  Return
```

Algorithm: Dequeue (Deletion) from Circular Queue

- Algorithm Dequeue(Queue, N)

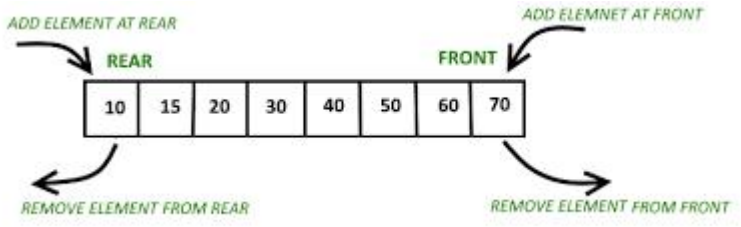
```
1.  If (Front == -1) Then
        Print "Queue Underflow"
        Return
2.  DeletedItem = Queue[Front]
3.  If (Front == Rear) Then   // Single element case
        Front = -1
        Rear = -1
    Else
        Front = (Front + 1) % N
4.  Return DeletedItem
```

DEQUES(Double ended queues)

- A deques (pronounced either “deck” or “dequeue”) is a linear list in which elements can be added or removed at either end but not in the middle.
- There are two variation of deque

1. **Input-restricted queue:** Deque which allows insertions at only one end of the list but allows deletion at both ends of the list.
2. **Output-restricted queue:** Deque which allows deletion at only one end of the list but allows insertion at both ends of the list.

- Deque can assume many of the characteristics of stacks and queues, it does not require the LIFO and FIFO orderings

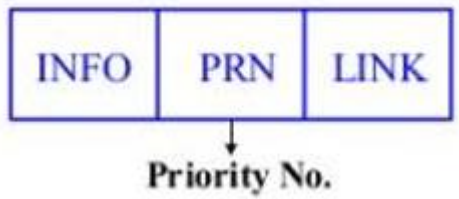


PRIORITY QUEUE:

- A priority queue is a collection of elements such that each elements has been assigned a priority and such that the order in which elements are deleted and processed comes from the following rules:
  1. Elements of higher priority is processed before any elements of lower priority
  2. Two elements with the same priority are processed according to the order in which they were added to the queue
- There are different ways a priority queue can be represented such as
  1. One-way List
  2. Multiple queue

One-Way List Representation of a Priority Queue

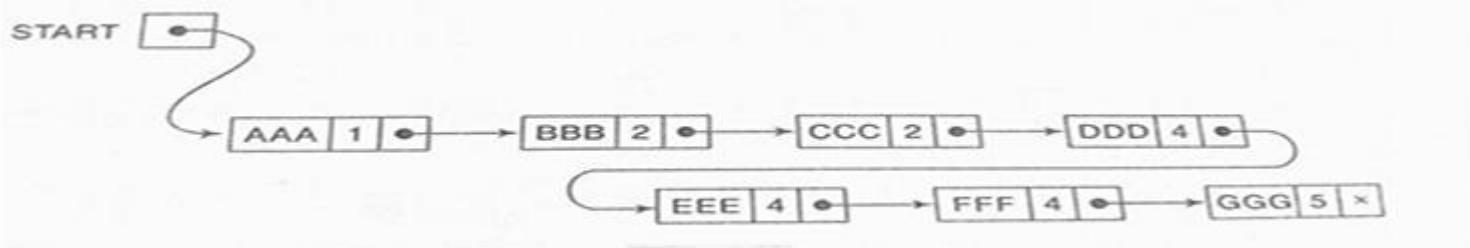
1. Each node in the list will contain three items of information: an information field INFO, a priority number PRN, and a link number LINK



2. A node X precedes a node Y in the list

   (a) when X has higher priority than Y or

   (b) when both have same priority but X was added to the list before Y



Using Multiple Queues

- Maintain separate queues for each priority level.
- Each queue can be a circular array with its own FRONT and REAR pointers.
- Insertions go to the queue corresponding to the element’s priority.
- Deletions check highest priority queue first and remove its front element.

Advantages:

- Fast insertion
- Easier to implement when priorities are limited

Disadvantages:

- Wastes memory if some priority queues are rarely used
- Deletion may require checking multiple queues

- Suppose we have a priority queue with priorities 3 > 2 > 1, and elements arrive as:

Element	Priority
A	2
B	3
C	2
D	1

One-way list representation:

After insertion: B → A → C → D

Deletion order: B, A, C, D

Multiple queue representation:

Queue 3: B

Queue 2: A → C

Queue 1: D

Deletion order: B (Queue 3) → A (Queue 2) → C (Queue 2) → D (Queue 1)